BorderPulse — Session 4

UI Reality Check, Camera Setup, Buzzer Logic, Full Error Log & Antigravity Prompt

Covers: border-pulse-zeta.vercel.app + the linked GitHub repo, plus everything found in D:\hackelite across all prior sessions.

Note: the Filesystem connector to D:\hackelite disconnected partway through this session, before the camera module could be re-read live. Camera findings below combine what was already verified in earlier sessions with general diagnosis — Section 3 tells Antigravity exactly what to re-confirm live.

1. Most important finding first: the Vercel link is not your app

border-pulse-zeta.vercel.app and github.com/kusoumya2006-netizen/BorderPulse are a SEPARATE, STATIC, DEPENDENCY-FREE marketing/concept page — not the working system in D:\hackelite.

Confirmed directly from the repo's own README: it's plain `index.html / app.js / style.css`, described as "the scroll-driven BorderPulse concept," run with `python3 -m http.server` — no backend, no FastAPI, no camera feed, no Supabase, no ESP32 connection of any kind. It's a single-page animated landing site.

This matters because the page displays specific numbers as if they were live telemetry: **TRACK_017 · 91.4% · an "IR SENSOR" reading 76%** (your real hardware has no IR/thermal sensor — that was explicitly scoped as a future-only interface in prior audits), plus static stats like "68.4% false-alarm reduction," "420ms avg. detection latency," "1,284 tracks today," "05 cameras online." None of this is wired to anything real — it's all hardcoded copy for visual effect.

Why this is a genuine risk for judging, not just a technicality

If this page is shown to judges as if it reflects the live system's performance, a technical judge who asks "where does the 68.4% come from?" or "show me the IR sensor" will find there's no answer — that's a credibility risk bigger than any code bug in this report. Recommended framing instead: present it explicitly as a concept/vision page for the pitch, separate from the live operational dashboard (Overview/LiveMonitor/Zones/etc. in the real frontend) which shows only real, currently-computed numbers. Two honest artifacts beat one dishonest-looking one.

If the goal is to make this real

- Replace the hardcoded numbers with a small fetch to the real backend's `/api/health` and `/api/events` endpoints, so "cameras online" and "alerts today" reflect actual state.
- Remove or clearly re-label the IR sensor tile — don't display a sensor reading that doesn't exist in the hardware.
- Keep the false-alarm-reduction and latency stats out entirely until you have real measured numbers from your own test suite (Section 15 of the earlier audit prompt covers exactly this methodology) — a specific, sourced number is more credible than an impressive but unverifiable one.

2. Recap: verified state of the real app (D:\hackelite), across all sessions

Item	Status	Note
Camera capture, YOLO+tracking, zone engine, fusion, decision engine, evidence capture, ESP32 client, heartbeat	REAL	All confirmed by direct code reading in session 1
Supabase service-role key	FIXED	Was a placeholder in session 1/2; confirmed real in session 3's .env rea
Ground vibration sensor (GPIO26)	REAL HARDWARE	Session 3: firmware does real digitalRead() with INPUT_PULLUP, confir
Radar sensor (GPIO27)	NOT CONNECTED	Firmware explicitly reports mode NOT_CONNECTED; backend still simu
Zone create / delete UI	WORKING	Session 3: two-layer canvas editor is correctly built; if it feels laggy, likely
Buzzer — manual test	WORKING	POST /test/buzzer confirmed reliable by user
Buzzer — real intrusion trigger	BUG — ROOT CAUSE FOUND	See Section 4. UI shows buzzer_active before hardware ever confirms it
WebSocket stream-loop crash from user's log	LIKELY ALREADY FIXED	Current app.py's dead/ws handling is structurally correct; needs live repr
Port 8000 bind error from user's log	ENVIRONMENT, NOT CODE	Previous BorderPulse process still running; see Section 6 for the fix
YOLO model	UPGRADE AVAILABLE	YOLO26n confirmed real (Jan 2026 Ultralytics release), ~43% faster tha

3. Backend camera setup — diagnosis

Important caveat: the filesystem connector to your PC disconnected before backend/camera/capture.py and config.py could be re-read fresh for this session, so the specifics below combine what was already confirmed in session 1 (camera capture is a real, working module: dedicated capture thread, bounded frame queue, DirectShow fallback for Windows, FPS tracking) with the most common, concrete failure causes for that exact architecture. Antigravity should re-verify against the live file before applying any fix, since the code may have changed since session 1.

Most likely causes, in order of probability for a Windows laptop setup

- **CAMERA_INDEX mismatch.** .env's CAMERA_INDEX=0 assumes the built-in webcam is device 0 — on a laptop with a virtual camera driver installed (e.g. from OBS, a phone-as-webcam app, or a Bluetooth headset with video passthrough), index 0 can silently point at the wrong device or a device that returns no frames. Try CAMERA_INDEX=1 or 2 and confirm which index actually opens with a live image.
- **Camera already in use by another application.** Windows only allows one process to hold most webcams at a time — if a video call app, another BorderPulse instance, or the browser has a tab with camera permission open, cv2.VideoCapture will open successfully but return black/empty frames or fail silently depending on the backend.
- **DirectShow (CAP_DSHOW) backend selection.** OpenCV on Windows defaults to Microsoft Media Foundation (MSMF) unless the code explicitly requests CAP_DSHOW — MSMF has known slow-open and black-frame bugs with some webcam drivers. If capture.py already has a DSHOW fallback (per session 1's finding), confirm it's actually being reached, not just defined — a try/except that silently swallows the MSMF failure without falling through would explain a "camera not working" symptom with no visible error.
- **Resolution/format negotiation failure.** If the code requests a specific resolution (e.g. 1280x720 from the earlier logs) that the physical camera doesn't support at the requested FPS, some drivers return failure rather than a supported fallback resolution.

What to add to make this diagnosable going forward

Camera failures are currently probably visible only as a generic health-check False. Add explicit logging at camera startup: which backend (MSMF/DSHOW) was actually opened, the negotiated resolution/FPS returned by the driver (not just what was requested), and the first-frame-read success/failure with the specific cv2 error if any. Right now the log likely just says CAMERA [FAIL] with no further detail — that's not enough to diagnose remotely or quickly during a live demo.

4. Buzzer logic — when it fires, and the confirmed root cause bug

4.1 — Conditions for the buzzer to fire (decision engine, backend/decision/engine.py)

- Detection class must be 'person', AND
- The person's bounding-box bottom-center (feet) point must be inside an enabled restricted zone, AND
- Either: the same track is confirmed inside the zone for PERSON_CONFIRMATION_FRAMES consecutive stable frames (default 4) — a sudden bbox jump resets this counter — OR the high-confidence fast path applies (confidence >= YOLO_HUMAN_HIGH_CONFIDENCE, default 0.85), which skips the frame-count wait but still requires feet-inside-zone, AND
- A cooldown window (EVENT_COOLDOWN_SECONDS, default 10s) has elapsed since the last trigger for that track.

4.2 — What happens in backend/app.py the instant should_alarm becomes True

```
_buzzer_active_state = True # set IMMEDIATELY -- before the ESP32 call runs
future = loop.run_in_executor(None, esp32_client.trigger_alarm, event_reason)
future.add_done_callback(_log_esp32_result) # logs success/failure ASYNCHRONOUSLY, afterward
```

4.3 — CONFIRMED ROOT CAUSE of "software alert fires but buzzer doesn't reliably activate"

`_buzzer_active_state` — the exact flag the frontend reads as `buzzer_active` in the WebSocket stream — is set to True the instant the decision is made, before `esp32_client.trigger_alarm()` has actually run or returned anything. Two concrete failure paths from there:

- trigger_alarm() refuses to even attempt the HTTP call if the cached status.online is False at that instant. status.online only updates via a heartbeat thread every ESP32_HEARTBEAT_INTERVAL seconds (5s default) — on a phone-hotspot connection (which this ESP32 is on), a brief dropout right at heartbeat time leaves status.online falsely False for up to 5 seconds, during which a real intrusion event would silently skip sending the command entirely, while the UI already claims the buzzer is active.
- Even when the HTTP call is attempted and genuinely fails (timeout, non-200, connection reset), the failure is logged but nothing reverts _buzzer_active_state back to False — so the UI keeps showing the alarm as active regardless of what actually happened at the hardware.

4.4 — Required fix

- Do not set _buzzer_active_state=True optimistically. Track a separate 'command sent' flag at request time, and only flip the UI-facing buzzer_active to True inside the done-callback once fut.result() confirms the ESP32 actually returned HTTP 200.
- Expose three distinct states to the frontend, not one boolean: ALERT DECIDED → ALARM COMMAND SENT → BUZZER ACKNOWLEDGED. Never claim the buzzer is on unless the acknowledged state is true.
- Let trigger_alarm() attempt the real HTTP call even when the cached heartbeat status is stale — an actual request response is a more current signal than a heartbeat that could be up to 5 seconds old. Keep the heartbeat for the passive ONLINE/OFFLINE badge, just don't let it gate a real alarm attempt.

4.5 — When the buzzer turns off

Software side: once no tracked person remains in ALARM_ACTIVE/EVIDENCE_CAPTURE/EVENT_ACTIVE state (left the zone, or lost for TRACK_LOSS_GRACE_SECONDS), the backend sends POST /alarm/stop once. Independently, the ESP32 firmware's own checkAlarmTimeout() turns the buzzer off once the duration_ms from the original /alarm request elapses — a correct hardware-side safety net, leave this as-is.

5. Restricted zone — create / delete (already working, quick reference)

Create

Zones page → "+ Draw Zone" → click 3+ points directly on the live camera image → type a name → "Save (N pts)". Coordinates are normalized 0-1 in camera space with correct letterbox/pillarbox handling, then POSTed to /api/zones and persisted to Supabase.

Delete

Red "Delete" button next to the zone in the list → confirm the browser prompt → DELETE /api/zones/{id} removes it from both the live engine and Supabase.

If it still feels laggy

Don't rewrite this editor — it's already well-built (RAF-driven interaction layer, no React state thrash on mouse move). The likely real cost is the camera reference layer fully re-decoding a JPEG on every single WebSocket frame via a new Image() object even while not actively drawing. Throttle or skip that decode when the zone editor tab isn't focused, or when a new frame hasn't meaningfully changed.

One real gap found in session 3, still open

backend/api/zones.py's create_zone() swallows Supabase write failures silently (`except Exception: pass`) — a zone can appear saved in the UI while never actually persisting to the cloud. Surface this failure instead of hiding it.

6. Full error log, consolidated across every session

6.1 — WebSocket stream-loop crash (from the user's terminal log)

```
[STREAM_LOOP] Error in loop: name 'dead' is not defined
UnboundLocalError: local variable 'ws' referenced before assignment
NameError: name 'dead' is not defined
```

The version of backend/app.py read in session 3 already has this structured correctly (dead = [] defined before the loop, ws properly scoped as the loop variable) — this may already be fixed since the log was captured. Reproduce live to confirm; if it still occurs, harden defensively with `for ws in list(_active_ws_clients):` to protect against concurrent mutation regardless.

6.2 — Port already in use

```
ERROR: [Errno 10048] error while attempting to bind on address ('0.0.0.0', 8000):
only one usage of each socket address (protocol/network address/port) is normally permitted
```

Not a code bug — a previous BorderPulse backend process is still holding port 8000. Fix right now: PowerShell → `netstat -ano | findstr :8000` → note the PID in the last column → `taskkill /PID <pid> /F` → restart. Also add a pre-bind check to the startup script so this fails with a clear message instead of a raw OSError after "Application startup complete" (which is confusingly printed before the bind failure, since the FastAPI lifespan finishes before uvicorn actually attempts the socket bind).

6.3 — Startup health banner is stale for the ground sensor

backend/app.py's lifespan() hardcodes `"ground": "SIMULATED"` in the startup health dict before the ESP32 heartbeat check ever runs — so the one-time startup log always prints GROUND SIMULATED even now that the ground sensor is confirmed real hardware. The live per-frame stream payload computes this correctly elsewhere; only the one-line startup banner is wrong. Compute it the same way, after the heartbeat check completes.

6.4 — Supabase zone-save failures silently swallowed

See Section 5 — bare except Exception: pass in create_zone() hides real persistence failures from both the user and the logs.

6.5 — Buzzer state misreported to the UI

See Section 4.3/4.4 — this is the highest-priority fix in this entire report.

7. Master prompt — paste into Antigravity

This section is written to be copy-pasted directly. It references the two prior session files already saved in the project root, and adds everything new from this session.

```
Read ANTIGRAVITY_FULL_AUDIT_AND_FIX_PROMPT.md and
ANTIGRAVITY_SESSION3_BUZZER_ZONES_YOLO26_ERRORS.md in the project root first -- both are still valid.
This session adds:

1. LANDING PAGE VS REAL APP: border-pulse-zeta.vercel.app / the kusoumya2006-netizen/BorderPulse repo
is a separate static concept page with hardcoded fake numbers (IR sensor that doesn't exist in our
hardware, fake false-alarm-reduction/latency/track-count stats). Do not present these numbers as real
results. Either wire its key tiles to real backend endpoints (/api/health, /api/events) or clearly
label it as a concept/pitch page separate from the live dashboard. Remove the IR sensor tile or label
it 'future roadmap, not present hardware' -- do not display a live-looking reading for a sensor that
isn't physically present.

2. CAMERA SETUP: re-verify backend/camera/capture.py and .env's CAMERA_INDEX live -- I could not re-read
this file in my last session due to a disconnected tool. Check in this order: (a) which CAMERA_INDEX
values actually open a live, non-black frame on this machine, (b) confirm whether the code's DSHOW
fallback (if present) is actually reached when MSMF fails, or silently swallowed, (c) add explicit
startup logging: which backend opened, negotiated resolution/FPS, and the specific cv2 error on
first-frame-read failure -- do not leave camera failures as a bare CAMERA [FAIL] with no detail.

3. BUZZER ROOT CAUSE (highest priority fix in this session): in backend/app.py, _buzzer_active_state is
set to True optimistically before esp32_client.trigger_alarm() has run or returned. Fix: only set the
UI-facing buzzer_active True inside the executor's done-callback once fut.result() confirms a real
HTTP 200 from the ESP32. Expose three distinct states to the frontend: ALERT DECIDED, ALARM COMMAND
SENT, BUZZER ACKNOWLEDGED. Also let trigger_alarm() attempt the real HTTP call even if the cached
heartbeat status.online is stale, since a live request result is more current truth than a heartbeat
up to 5 seconds old -- keep the heartbeat only for the passive online/offline badge.

4. Fix the stale 'GROUND SIMULATED' startup banner (Section 6.3) to compute after the ESP32 heartbeat
check, matching the already-correct live per-frame logic.

5. Surface Supabase zone-save failures in backend/api/zones.py's create_zone() instead of swallowing
them with a bare except -- return a clear success/failure signal to the frontend.

6. Confirm live whether the WebSocket dead/ws crash from the earlier log still reproduces. If not, still
harden with list(_active_ws_clients) when iterating for broadcast, defensively.

7. Port-8000-in-use is an environment issue, not code -- just add a clear pre-bind error message; don't
spend real engineering time on it beyond that.

Work through items in priority order: 3, then 2, then 1, then 4/5/6/7. Fix one at a time, test, report
before moving to the next. Do not rebuild any working module. Report back with real log output proving
the buzzer fix works end to end on an actual test before touching anything else.
```

This report reflects a live read of border-pulse-zeta.vercel.app and its linked GitHub repo at generation time, combined with the D:\hackelite codebase as verified across sessions 1-3. Camera-module specifics in Section 3 need live re-confirmation per the caveat there.